Seminar 1 - Intro Java (recapitulare) & Colectii si Clase Generice

Cuprins

1. Sintaxa & Tipuri de date
2. Structuri de control
3. Concepte POO (Programare Orientata Obiect)
4. Set (HashSet) si List (ArrayList) & Stream
5. Comparable si Comparator
6. Clase Generice
7. Exercitii

> ⚠️ NOTA
>
> Fiecare student va realiza exemplele de mai jos pe calculatorul lui si de asemenea exercitiile de la final. Tot codul sursa pentru exercitiile individuale de la finalul documentului va fi incarcat pe online.ase.ro in sectiunea dedicata pana la finalul seminarului in vederea obtinerii unui punctaj.

1. Sintaxa & Tipuri de date

- Java are doua tipuri de variabile:
  - *primitive*
  - *obiecte*

```
// Primitive — stocate direct in stack
int age = 25;
long salary = 100_000L;
double rate = 3.14;
boolean active = true;
char grade = 'A';

// Referinte de obiecte — stocate in heap
String name = "Alice";
Integer boxedAge = 25; // primitive incapsulate in obiecte (clase dedicate)
(Integer incapsuleaza int)
```

- Java are doua mecanisme automate de conversie pentru tipurile de date primitive si wrapper-ele lor clase:
  - *autoboxing* - de la primitiv la obiect
  - *unboxing* - de la obiect la primitiv

```
List<Integer> numbers = new ArrayList<>();
numbers.add(42);        // autoboxing: int -> Integer
int x = numbers.get(0); // unboxing: Integer -> int
```

> ⚠️ Atentie!
>
> `==` aplicat la tipuri primitive va compara `valoarea`, insa aplicat la obiecte va compara `referinta` acestora si nu valoarea. Intotdeauna pentru compararea obiectelor se va folosi metoda `.equals()`.

- Exemplu:

```
Integer a = 200;
Integer b = 200;
System.out.println(a == b);      // false! (instante diferite)
System.out.println(a.equals(b)); // true
```

- `var` keyword. Lasa compilatorul sa atribuie tipul de data al variabilei. Util cand avem tipuri de date complexe si nu vrem sa avem foarte mult cod repetat. De obicei este folosit atunci cand in partea dreapta a egalului este evident tipul de data, altfel ar trebui sa omitem folosirea lui in exces, pentru ca poate aduce si confuzii programatorilor.

```
var name = "Alice";         // String
var age = 25;               // int
var list = new ArrayList<String>(); // ArrayList<String>

// var NU ESTE dinamic — blocul de mai jos nu va compila
var x = 10;
x = "hello"; // EROARE: tipuri incompatibile
```

2. Structuri de control

- Structuri alternative
  - if - else if
  - switch
  - operator ternar
- Structuri repetitive
  - while / do while
  - for loop / for-each

```
// if / else
int score = 75;
if (score >= 90) {
    System.out.println("A");
} else if (score >= 70) {
    System.out.println("B");
} else {
    System.out.println("C");
}

// switch
int day = 4;
switch (day) {
case 1:
    System.out.println("Monday");
    break;
case 2:
    System.out.println("Tuesday");
    break;
case 3:
    System.out.println("Wednesday");
    break;
case 4:
    System.out.println("Thursday");
    break;
case 5:
    System.out.println("Friday");
    break;
case 6:
    System.out.println("Saturday");
    break;
case 7:
    System.out.println("Sunday");
    break;
}

// ternary operator
String isPassed = grade >= 5 ? "YES" : "NO";

// for loop
for (int i = 0; i < 5; i++) {
```

```java
    System.out.println(i);
}

// enhanced for (for-each)
int[] numbers = {1, 2, 3, 4, 5};
for (int n : numbers) {
    System.out.println(n);
}

// while
int count = 0;
while (count < 3) {
    System.out.println(count++);
}
```

3. Concepte POO (Programare Orientata Obiect)

- **Incapsulare**
  - se refera la incapsularea datelor (variabilelor) si a metodelor impreuna intr-o clasa restrictionand accesul direct la a modifica datele
  - apar notiunile de `getters` si `setters` pentru variabilele clasei

```
class Person {
    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

- **Mostenire**
  - presupunea capabilitatea unei clase (sub clasa) de a mosteni proprietati (variabile) si metode de la clasa parinte (super clasa)
  - relatia se poate citi ca `este un / o` si se defineste prin keyword-ul `extends`
  - o clasa poate extinde doar o singura clasa si poate implementa mai multe interfete

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

Dog dog = new Dog();
dog.eat(); // Eating - fiecare animal mananca
dog.bark() // Barking - nu toate animalele latra, in cazul nostru cainele da
```

- **Polimorfism**
  - prin acest mecanism metodele se pot comporta diferit in functie de tipul de obiect care le apeleaza

1. **Method Overloading (compile-time)**

   - presupune supraincarcarea unei metode cu acelasi nume, dar avand numar de parametrii diferiti si/sau tipuri de date ale acestora diferite

```
class MathUtils {
    int add(int a, int b) {
        return a + b;
    }

    float add(float a, float b, float c) {
        return a + b + c;
    }
    // metodele au acelasi nume, insa compilatorul va stii pe care sa o
apeleze in functie de numarul si tipul parametrilor
}
```

2. **Method Overriding (runtime)**

   - presupune suprascrierea unei metode din clasa parinte / interfata pentru a implementa un comportament specific clasei respective
   - in practica se foloseste adnotarea `@Override` pentru a instiinta compilatorul sa verifice ca efectiv suprascriem ceva

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

Animal dog = new Dog(); // obiectul este din clasa Animal, insa instanta este
de tip Dog
Animal someAnimal = new Animal(); // obiectul este din clasa Animal si
instanta la fel
dog.sound(); // dog este o instanta de tipul Dog in spate, deci va apela
metoda din clasa Dog - "Dog barks"
```

```
someAnimal.sound(); // someAnimal este o instanta de tipul Animal, deci va
apela metoda din clasa Animal - "Animal makes sound"
```

- **Abstractizare**
  - inseamna ascunderea detaliilor complexe de implementare si expunerea doar a functionalitatilor necesare obiectelor
  - aici apare notiunea de `clase abstracte` si `interfete`

```
// clasa abstracta

abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

// interfata

interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

	Interfata	**Clasa abstracta**
Mostenire multipla	Da (o clasa poate implementa mai multe)	Nu (doar una)
Campuri (stare)	Nu (doar constante `static final`)	Da
Constructor	Nu	Da
Metode concrete	Da (prin `default`, Java 8+)	Da
Metode abstracte	Da (implicit)	Da (explicit cu `abstract`)
Cand o folosesti	Definesti un **comportament / contract**	Impartasesti o **implementare comuna**

4. Set (HashSet) si List (ArrayList) & Stream

- `Collection` este interfata principala pentru colectiile din Java
- Din ea deriva `Set` si `List` (si nu numai)
  - `Set` are implementarea cea mai cunoscuta `HashSet`. Nu poate contine duplicate si nu pastreaza ordinea.
  - `List` are implementarea cea mai cunoscuta `ArrayList` care functioneaza ca un array dinamic, redimensionat automat. Poate contine duplicate si elementele sunt accesate dupa index
- SET

```
Set<String> cars = new HashSet<>();
cars.add("Mercedes");
cars.add("Audi");
cars.add("BMW");

// mod natural de parcurgere
for (String car : cars) {
    System.out.println(car);
}

if(!cars.add("Audi")){ // element duplicat, nu se adauga
    System.out.println("Audi exista deja in Set!");
};

// parcurgere si cu iterator
Iterator<String> iterator = cars.iterator();
while(iterator.hasNext()){
    System.out.println("Masina " + iterator.next());
}

if(cars.contains("Toyota")){
    System.out.println("Exista Toyota!");
} else {
    System.out.println("Nu exista Toyota, o vom adauga direct!");
    cars.add("Toyota");
}

// stream like
cars.forEach(System.out::println);

System.out.println("Stergere Toyota");
if(!cars.remove("Toyota")){
    System.out.println("Toyota nu exista!");
}

System.out.println("Dimensiune: " + cars.size());
System.out.println("Verificare daca e gol: " + cars.isEmpty());

System.out.println("Stergere set");
cars.clear();
System.out.println("Verificare daca e gol: " + cars.isEmpty());
```

```java
// repopulare cars
cars.add("Mercedes");
cars.add("Audi");
cars.add("BMW");

Set<String> anotherCars = new HashSet<>();
anotherCars.add("Dacia");
anotherCars.add("BMW");
anotherCars.add("Renault");

// reuniune
Set<String> union = new HashSet<>(cars);
union.addAll(anotherCars);
System.out.println("\nREUNIUNE");
union.forEach(System.out::println);

// intersectie
Set<String> intersection = new HashSet<>(cars);
intersection.retainAll(anotherCars);
System.out.println("\nINTERSECTIE");
intersection.forEach(System.out::println);

// diferenta
Set<String> diff = new HashSet<>(cars);
diff.removeAll(anotherCars);
System.out.println("\nDIFERENTA");
diff.forEach(System.out::println);
```

> ⚠️ Atentie la obiecte custom!
>
> In cazul utilizarii set-urilor pe obiecte proprii trebuie neaparat sa suprascriem metodele `.equals()` si `.hashCode()` pentru a functiona cum trebuie. In cazul in care nu facem acest lucru, atunci HashSet spre exemplu nu va detecta corect duplicatele.

```java
public class Car {
    private final String name;

    public Car(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Car car)) return false;
        return name.equals(car.name);
    }

    @Override
    public int hashCode() {
```

```
        return name.hashCode();
    }
}

Set<Car> carsSet = new HashSet<>();
Car c1 = new Car("Audi");
Car c2 = new Car("BMW");

carsSet.add(c1);
carsSet.add(c2);

carsSet.forEach(System.out::println);

carsSet.add(new Car("Audi")); // duplicat conform denumirii masinii, nu se va
adauga

System.out.println("Dupa adaugare Audi (DUPLICAT)");
carsSet.forEach(System.out::println);
```

- LIST

```
import java.time.LocalDate;

public class Transaction {
    private final String id;
    private final String type;
    private final float amount;
    private final LocalDate date;

    public Transaction(String id, String type, float amount, LocalDate date){
        this.id = id;
        this.type = type;
        this.amount = amount;
        this.date = date;
    }

    public Transaction(String id, String type, float amount){
        this.id = id;
        this.type = type;
        this.amount = amount;
        this.date = LocalDate.now();
    }

    public String getId() {
        return this.id;
    }

    public float getAmount() {
        return this.amount;
    }

    public String getType() {
```

```java
        return this.type;
    }

    public LocalDate getDate() {
        return this.date;
    }

    @Override
    public String toString() {
        return "Transaction{" +
                "id='" + id + '\'' +
                ", type='" + type + '\'' +
                ", amount=" + amount +
                ", date=" + date +
                '}';
    }
}

// initializare lista tranzactii
private static List<Transaction> loadTransactions(){
    List<Transaction> transactions = new ArrayList<>();

    for(int i = 1; i <= 15; i++){
        String id = "TXN-" + i;
        String type = i%3 == 0 ? "DEBIT" : "CREDIT";
        float amount = (float) Math.round(RandomGenerator.getDefault().nextFloat()
* 10_000) / 100;
        LocalDate date = LocalDate.of(2026, 4, i % 3 == 0 ? i : i + 1);

        transactions.add(new Transaction(id, type, amount, date));
    }

    return transactions;
}

// main

List<Transaction> transactions = loadTransactions();

Transaction transaction = transactions.get(0); // prima tranzactie
System.out.println(transaction);

System.out.println("Nr tranzactii: " + transactions.size());

System.out.println("\nPARCURGERE CU FOR");
for(int i = 0; i < transactions.size(); i++){
    System.out.println(transactions.get(i));
}

System.out.println("\nPARCURGERE CU FOR-EACH");
for(Transaction t : transactions){
    System.out.println(t);
}
```

```
System.out.println("\nPARCURGERE STREAM LIKE");
transactions.forEach(System.out::println);

// adaugare tranzactie in lista
Transaction txn = new Transaction("TXN-" + (transactions.size() + 1), "DEBIT",
104.32f);
transactions.add(txn);

System.out.println("Tranzactia adaugata: " + transactions.get(transactions.size()
- 1));

if(transactions.remove(txn)){ // stergere
    System.out.println("\nTranzactie stearsa cu succes!");
    System.out.println("\nDupa stergere");
    transactions.forEach(System.out::println);
}
```

- Stream

```
List<Transaction> transactions = loadTransactions();

// FILTER
List<Transaction> debits = transactions.stream().filter(t ->
t.getType().equals("DEBIT")).toList();
System.out.println("DEBITS");
debits.forEach(System.out::println);

// MAP (transformare)
List<Float> amounts = transactions.stream().map(Transaction::getAmount).toList();
System.out.println("\nAMOUNTS");
amounts.forEach(System.out::println);

// LIMIT si SKIP
transactions.stream().skip(2).limit(2).forEach(System.out::println); //
tranzactiile 3 si 4 (primele 2 skipped)

// COUNTS
long count = transactions.stream().filter(t ->
t.getType().equals("CREDIT")).count();
System.out.println("\nCREDITS count");
System.out.println(count);

// REDUCE v1
float balance = transactions.stream().map(t -> {
    if(t.getType().equals("DEBIT")){
        return - t.getAmount();
    }

    return t.getAmount();
}).reduce(0f, Float::sum);

System.out.println("\nBALANCE v1");
System.out.println(balance);

// REDUCE v2
float balance2 = transactions.stream().reduce(
        0f,
        (sum, t) ->
                t.getType().equals("DEBIT") ? sum - t.getAmount() : sum +
t.getAmount(), Float::sum
);

System.out.println("\nBALANCE v2");
System.out.println(balance2);

System.out.println("\n");

// ANY / ALL / NONE MATCH
boolean anyBig = transactions.stream()
```

```java
        .anyMatch(t -> t.getAmount() > 95);

boolean allDebits = transactions.stream()
        .allMatch(t -> t.getType().equals("DEBIT"));

boolean noneNegative = transactions.stream()
        .noneMatch(t -> t.getAmount() < 0);

System.out.println("ANY BIG: " + anyBig);
System.out.println("ALL DEBITS: " + allDebits);
System.out.println("NONE NEGATIVE: " + noneNegative);

// GROUP BY
Map<String, List<Transaction>> grouped = transactions.stream()
        .collect(Collectors.groupingBy(Transaction::getType));

grouped.forEach((type, list) -> {
    System.out.println(type + ":");
    list.forEach(System.out::println);
});

// OPTIONAL — gasire prima tranzactie DEBIT cu amount > 1000
Optional<Transaction> bigDebit = transactions.stream()
        .filter(t -> t.getType().equals("DEBIT") && t.getAmount() > 1000)
        .findFirst();

bigDebit.ifPresentOrElse(
        t -> System.out.println("Primul DEBIT mare: " + t),
        () -> System.out.println("Nu exista niciun DEBIT cu amount > 1000")
);

// OPTIONAL — extrage amount-ul sau returneaza o valoare default
float bigDebitAmount = bigDebit
        .map(Transaction::getAmount)
        .orElse(0f);

System.out.println("Amount (sau 0 daca nu exista): " + bigDebitAmount);
```

5. Comparable si Comparator

- `Comparable` este o interfata din `java.lang` si este folosita la definirea `ordinii naturale` a unui obiect. Clasele vor implementa aceasta interfata si vor suprascrie metoda `int compareTo(T o)` unde `T` este generic. Este folosita implicit de `Collections.sort()`, `TreeSet` sau `TreeMap` (care mentin ordinea) si etc.
- `Comparator` este o interfata din `java.util` si este folosita la definirea de `ordini alternative` ale unui obiect. Clasa nu se modifica, putem avea mai multe criterii si este foarte flexibil.
- Comparable

```
public class Transaction implements Comparable<Transaction> {
    private String id;
    private String type;
    private float amount;
    private LocalDate date;

    // constructors, setters, getters, toString

    // din interfata
    @Override
    public int compareTo(Transaction o) {
        return Float.compare(this.amount, o.amount);
    }
}

// main

List<Transaction> transactions = loadTransactions();

transactions.forEach(System.out::println);

System.out.println("\nSORTED");
Collections.sort(transactions); // reverse() in loc de sort() pentru ordinea
inversa
transactions.forEach(System.out::println);
```

- Comparator

```
System.out.println("\nSORTED");
transactions.sort(Comparator.naturalOrder()); // echivalent exemplului de mai sus
transactions.forEach(System.out::println);

System.out.println("\nSORTED");
transactions.sort(Comparator.comparingDouble(Transaction::getAmount)); //
echivalent exemplului de mai sus
transactions.forEach(System.out::println);
```

```
System.out.println("\nSORTED REVERSE");
transactions.sort(Comparator.reverseOrder());
transactions.forEach(System.out::println);

System.out.println("\nSORTED BY DATE REVERSE AND AMOUNT");
transactions.stream()
        .sorted(Comparator
                .comparing(Transaction::getDate, Comparator.reverseOrder()) //
date DESC
                .thenComparing(Transaction::getAmount))                     //
amount ASC
        .forEach(System.out::println);


System.out.println("\nMAX si MIN");
transactions.stream().max(Comparator.naturalOrder()).ifPresent(t ->
System.out.println("MAX tnx: " + t));
transactions.stream().min(Comparator.naturalOrder()).ifPresent(t ->
System.out.println("MIN tnx: " + t));

System.out.println("\nMAX amount si MIN amount");
transactions.stream().map(Transaction::getAmount).max(Float::compare).ifPresent(am
ount -> System.out.println("Max amount: " + amount));
float min =
transactions.stream().map(Transaction::getAmount).min(Float::compare).orElse(0.f);
System.out.println("Min amount: " + min);
```

6. Clase Generice

- Genericele reprezinta un mecanism prin care se pot defini clase, interfete si metode care pot lucra cu tipuri de date diferite, fara a pierde siguranta tipurilor.
- Principalul avantaj este reutilizarea, nu trebuie rescris codul de fiecare data pentru fiecare tip de data (String, Integer).
- Acestea functioneaza doar cu clase, nu si cu primitive (int, char, boolean).
- Scopuri:
  - siguranta tipurilor de date. Erorile sunt aruncate la compilare, nu la runtime
  - eliminarea casting-ului
  - reutilizare de cod
- In loc de specificarea tipului de data efectiv se va specifica un parametru de tip `T` sau `K si V`, cele mai folosite in practica.

```java
public class Pair<K, V> {
    private final K key;
    private final V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return this.key;
    }

    public V getValue() {
        return this.value;
    }

    @Override
    public String toString() {
        return this.key + " -> " + this.value;
    }
}

// main

Pair<String, Integer> pair1 = new Pair<>("Sabin", 24);
System.out.println(pair1.getKey() + " are " + pair1.getValue() + " de ani.");
System.out.println(pair1.toString());

Pair<String, List<String>> pair2 = new Pair<>("Sabin", List.of("Math", "Computer
science"));
System.out.println(pair2.toString());
System.out.println(pair1.getKey() + " --- numar materii " +
pair2.getValue().size());
```

7. Exercitii

A. Sistem de notificari

Defineste o interfata `Notifiable` cu metodele:

```
void send(String message);
String getChannel();
```

1. Implementeaza trei clase concrete: `EmailNotification`, `SmsNotification`, `PushNotification`, fiecare cu un camp specific (ex. adresa email, numar telefon, device token).
2. Creeaza o clasa abstracta `BaseNotification` care:
   - implementeaza interfata `Notifiable`
   - contine un camp `timestamp` de tip `LocalDateTime`, setat automat la creare
   - contine o metoda concreta `log()` care afiseaza: `[channel] [timestamp] message`
3. Toate cele trei clase extind `BaseNotification` si suprascriu `send()`, apeland si `log()` in interior.
4. Intr-un `List<Notifiable>`, adauga instante mixte si trimite acelasi mesaj tuturor. Demonstreaza polimorfismul.

> HINT: O clasa poate extinde o clasa abstracta SI implementa o interfata simultan. Daca `BaseNotification` deja implementeaza `Notifiable`, subclasele mostenesc asta automat — nu trebuie sa scrii `implements Notifiable` din nou pe fiecare subclasa. Metodele `send()` si `String getChannel()` vor fi suprascrise direct in clasele concrete, chiar daca clasa abstracta implementeaza interfata. Neimplmentand metodele din interfata in clasa abstracta, vom fi obligati sa le implementam in clasele concrete, ceea ce ne si dorim.

B. Inventar de produse

Creeaza clasa `Product` cu campurile: `name`, `category` (String), `price` (double), `stock` (int).

1. Genereaza o lista de cel putin 5 produse din categorii mixte (ex. `"Electronics"`, `"Food"`, `"Clothing"`).
2. Folosind Stream, calculeaza si afiseaza:
   - valoarea totala a stocului (`price * stock`) per categorie, folosind `Collectors.groupingBy` + `Collectors.summingDouble`
   - top 3 produse dupa pret DESC
   - categoria cu cel mai mare numar de produse
   - lista produselor cu `stock == 0` — daca nu exista niciun produs cu stoc 0, afiseaza un mesaj clar
3. Sorteaza lista dupa categorie ASC, apoi dupa pret DESC in cadrul aceleiasi categorii.

> HINT: Pentru categoria cu cele mai multe produse: `groupingBy(category, counting())` returneaza un `Map<String, Long>` — aplica `.entrySet().stream().max(Map.Entry.comparingByValue()).map(Map.Entry::getKey)` care va returna `Optional<String>`.

C. Stiva generica cu istoric

Implementeaza o clasa generica `HistoryStack<T>` care functioneaza ca o stiva (LIFO) cu suport pentru undo.

Intern, foloseste doua `ArrayDeque<T>`:

- `stack` — stiva principala
- `history` — istoricul elementelor eliminate

Expune urmatoarele metode:

Metoda	Comportament
`push(T item)`	Adauga elementul pe stiva principala
`pop()`	Scoate varful stivei si il muta in `history`. Returneaza `Optional<T>`
`undo()`	Muta ultimul element din `history` inapoi pe stiva. Returneaza `Optional<T>` cu elementul restaurat
`peek()`	Returneaza `Optional<T>` cu varful stivei, fara sa il scoata
`size()`	Returneaza dimensiunea stivei principale
`historySize()`	Returneaza dimensiunea istoricului

Testeaza cu `HistoryStack<String>`:

- adauga 4 elemente
- fa `pop()` de doua ori si verifica `historySize()`
- fa `undo()` si verifica ca elementul a revenit pe stiva

> HINT: `ArrayDeque` are `push()` si `pop()` built-in (LIFO). Foloseste-le consistent pe ambele structuri.
> HINT: Pentru `Optional`: daca stack-ul / history-ul sunt goale atunci returnam `Optional.empty()`, altfel prelucram si returnam `Optional.of(item)`, unde `item` e de tipul `T`.

Asist. univ. **Sabin Tarbă**
Email: **sabin.tarba@csie.ase.ro**